

Due 12/6/2024

Please report what you have contributed to the project this term. Each student mentions the code they have written, or diagrams they have worked on. Please refer to your code in your Github account (what code each student has published in the Github account).

Team members and their contributions to the project:

Final Project Contributions

Josh Lollis

Project Overview

Total Lines of Code: ~12,014

- Backend: ~600 lines
- Frontend: ~11,414 lines
- Files Authored: 22
- Files Collaborated: 3

Project Leadership

Project Design & Architecture

- ✦ Designed complete system architecture
- ✦ Created architecture diagrams
- ✦ Designed all backend classes and functions
- ✦ Designed UI styling system
- ✦ Created comprehensive user stories

Project Management

- ✦ Created and maintained Kanban board
- ✦ Secured API integration
- ✦ Developed documentation & tutorials
- ✦ Researched technologies (Python, JS, QML)
- ✦ Authored reports and presentations

Technical Contributions

Backend Development (Python)

Fully Authored Backend Files (405 lines total)

Fully Authored Backend Files:

- `backendcontroller.py` (215 lines)

Core controller managing frontend-backend communication

- `cardprocessor.py` (145 lines)

Data transformation engine

- `inithandler.py` (20 lines)

Initialization management

- `main.py` (25 lines)

Application entry point

Collaborated Backend Files (225 lines total)

Collaborated Backend Files:

- `Backend.py` (110 lines)
 - *Personal contribution:* 67 lines
 - Query construction, documentation
- `SearchHandler.py` (45 lines)
 - *Personal contribution:* 4 lines
 - Core search logic
- `discoverhandler.py` (70 lines)
 - *Personal contribution:* 42 lines
 - Random selection algorithm

Frontend Development (QML)

💡 **Note:** All QML files below (22 total) were fully authored by me, representing ~11,414 lines of code

Major Components (~9,668 lines)

Major Components:

- `Search.qml` (2,162 lines)
- `Discover.qml` (2,252 lines)
- `Collection.qml` (2,415 lines)
- `AttackInfoBlock.qml` (1,661 lines)
- `MySearchFilterTools.qml` (1,178 lines)

UI Components

1. Card Information Display (~930 lines)

- 1. Card Information Display
 - `AbilityInfoBlock.qml` (280 lines)
 - `FlavorTextBlock.qml` (100 lines)
 - `SetLogoBlock.qml` (131 lines)
 - `SetSymbolBlock.qml` (136 lines)

- `TypeBlock.qml` (39 lines)
- `SubtypeBlock.qml` (132 lines)
- `Subtype.qml` (112 lines)

2. Interactive Components (~182 lines)

2. Interactive Components

- `CollectionButton.qml` (44 lines)
- `CollectionController.qml` (17 lines)
- `CompareButton.qml` (80 lines)
- `CompareController.qml` (41 lines)

3. Application Framework (~634 lines)

3. Application Framework

- `main.qml` (180 lines)
- `MySearchTools.qml` (159 lines)
- `SettingsWindow.qml` (11 lines)
- `Type.qml` (284 lines)

Detailed Technical Specifications

Backend Components

Core Controller System

- `backendcontroller.py` (215 lines):
 - Multi-threaded search request handling
 - Asynchronous discover feature implementation
 - Collection management (load/save)
 - Signal/slot communication with QML frontend
 - Error handling and debugging
 - JSON data serialization

Data Transformation Engine

- `cardprocessor.py` (145 lines):
 - Card data normalization
 - Ability processing
 - Attack data structuring
 - Type/subtype handling
 - Rules processing

Initialization Management

- `inithandler.py` (20 lines):
 - Set data retrieval
 - Frontend initialization support

Application Entry Point

- **main.py** (25 lines):
 - QML engine configuration
 - Backend controller integration
 - Style system setup
 - Application lifecycle management

Collaborated Backend Files

- **backend.py** (110 lines):
 - Implemented complete query construction logic (~62 lines)
 - Added class documentation (~5 lines)
 - *Remainder by others*: API configuration, base methods, and data fetching
- **searchhandler.py** (45 lines):
 - Implemented core search logic (~3 lines)
 - Added card processing integration (~1 line)
 - *Remainder by others*: Class structure, documentation, and error handling
- **discoverhandler.py** (70 lines):
 - Implemented core discover logic (~15 lines)
 - Developed random selection algorithm (~22 lines)
 - Added imports and utilities (~5 lines)
 - *Remainder by others*: Class structure and documentation

Frontend Components

Major Features

- **Search.qml** (2,162 lines):
 - Complex search parameter system
 - Dynamic filter management
 - Real-time card visualization
 - Multi-drawer architecture
 - Collection integration
 - Advanced card animations
 - Comprehensive UI components
 - Error handling system
- **Discover.qml** (2,252 lines):
 - Dual-drawer system architecture
 - Advanced card visualization
 - Dynamic data management
 - Real-time card updates
 - Complex animation system
 - Filter integration
 - Collection integration
 - Comprehensive error handling

- **Collection.qml** (2,415 lines):
 - Collection data management
 - Card visualization system
 - Drawer-based UI architecture
 - Complex state management
 - Real-time card updates
 - Search result processing
 - Collection persistence
 - Interactive card comparison
- **AttackInfoBlock.qml** (1,661 lines):
 - Energy cost management system
 - Dynamic image masking
 - Multi-state visual effects
 - Nested component architecture
 - Interactive property binding
 - Advanced layout management
- **MySearchFilterTools.qml** (1,178 lines):
 - Multi-type filter system
 - Dynamic button states
 - Image masking system
 - Interactive visual feedback
 - Property binding system
 - Layout management
 - Type-specific styling

UI Components

1. Card Information Display

- **AbilityInfoBlock.qml** (280 lines):
 - Property-driven styling system
 - Interactive text rendering
 - Multi-layered visual effects
 - Dynamic text scaling
 - Nested component architecture
- **FlavorTextBlock.qml** (100 lines):
 - Property-driven styling
 - Drop shadow text effects
 - Dynamic text scaling
 - Multi-layer visual system
 - Screen-like visual effects
- **SetLogoBlock.qml** (131 lines):
 - Property-driven styling system
 - Dynamic image loading

- Multi-layer visual effects
- Screen simulation system
- Drop shadow rendering
- Adaptive layout management
- **SetSymbolBlock.qml** (136 lines):
 - Property-driven styling system
 - Dynamic image loading
 - Multi-layer visual effects
 - Screen simulation system
 - Drop shadow rendering
 - Dynamic text scaling
- **TypeBlock.qml** (39 lines):
 - Property-driven styling system
 - Multi-type support
 - Dynamic layout management
 - Component reusability
 - Color scheme inheritance
- **SubtypeBlock.qml** (132 lines):
 - Property-driven styling system
 - Adaptive layout management
 - Dynamic visibility control
 - Complex layout calculations
 - Multi-block coordination
 - Flow-based arrangement
- **Subtype.qml** (112 lines):
 - Property-driven styling system
 - Screen simulation effects
 - Text rendering and scaling
 - Highlight management
 - Visual state handling

2. Interactive Components

- **CardFlow.qml** (151 lines):
 - Responsive grid layout system
 - Viewport-based card rendering
 - Interactive card scaling
 - Collection integration
 - Compare functionality
 - Memory optimization
- **CollectionButton.qml** (44 lines):
 - State-based visual feedback
 - Card collection integration
 - Dynamic opacity control
 - Image-based visualization
 - Event-driven interactions
- **CollectionController.qml** (17 lines):

- Collection data storage
- Add/Remove functionality
- Signal-based communication
- State persistence
- Console debugging support
- `CompareButton.qml` (80 lines):
 - Dual-state image toggle system
 - Left/Right comparison handling
 - Controller integration
 - Mouse interaction management
 - Visual state feedback
 - Dynamic image rotation
- `CompareController.qml` (41 lines):
 - Left/Right button state management
 - Signal-based communication
 - State clearing functionality
 - Debug logging support
 - Component coordination

3. Application Framework

- `main.qml` (180 lines):
 - Window configuration
 - Tab-based navigation system
 - Global color scheme management
 - Component layout management
 - Page integration and routing
 - State coordination
 - Collection persistence
- `MySearchTools.qml` (159 lines):
 - Search parameter management
 - Dynamic text input handling
 - Advanced button styling
 - Visual feedback system
 - Backend integration
 - Complex layout management
- `SettingsWindow.qml` (11 lines):
 - Multi-page settings interface
 - Theme management system
 - Developer tools integration
 - Data management interface
 - Navigation framework
 - Visual styling system
- `Type.qml` (284 lines):
 - Dynamic type mapping system
 - Complex image masking
 - Multi-layer visual effects

- Type-specific handlers
 - Advanced layout management
 - Property-driven styling
-